root-cause analysis tools, are good additions to a toolkit for testing. The "5 Whys" (Wikipedia, 2014a) use an iterative question-asking technique to explore root causes of a problem. Fishbone or Ishikawa diagrams (Wikipedia, 2014h) can be used for defect prevention and for identifying risks and potential pitfalls. After generating ideas in a brainstorming session, use techniques such as affinity diagrams or impact maps (see Figure 4-2) to organize the insights and potential experiments. Try different thinking tools to help elicit requirements and examples from customers. Drawing on a physical or virtual whiteboard enhances communication and creativity. Try it the next time your team gets together to identify impediments and experiments to chip away at them.

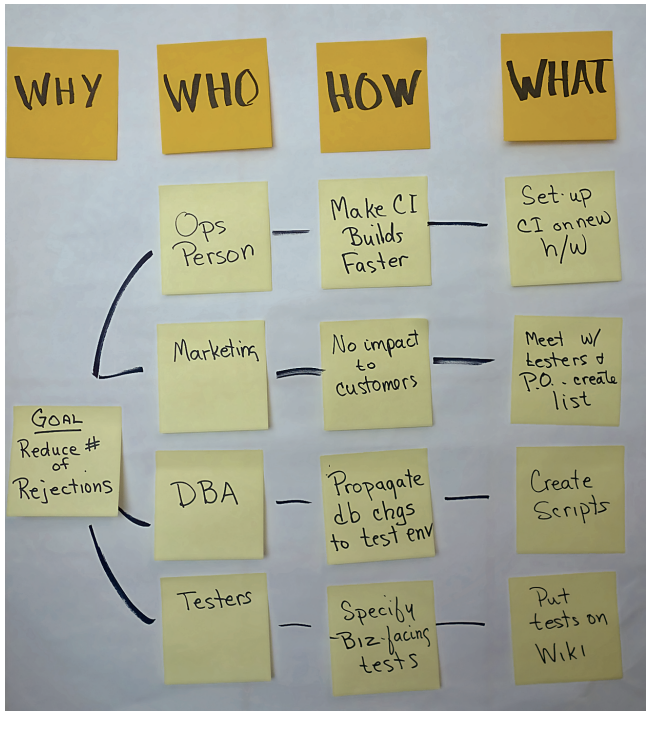

Figure 4-2 Sample impact map

Giving and Receiving Feedback

Giving and receiving feedback is almost an art form. Ideally, the giver's intention is to aid in the receiver's learning and to grow the relationship between them. We learned from Ellen Gottesdiener that feedback may actually say more about the giver than the receiver. The person giving feedback is basing it on his or her perceptions, and it is shaped by the person's emotions at the time of giving. Since we tend to give feedback about what matters to us, not what matters most to the other person, it is easy to miss context. Our tone, the words we choose, and our nonverbal communication may obscure our message.

Bear this in mind when you feel the urge to deliver your observations to another person. Think about how you'd want to receive the same feedback. Knowing how to keep the focus on the work, not on the person, is essential. Bug reporting is one way to provide feedback, but usually not the most effective. It takes time and practice to learn how to engage colleagues in a positive exchange when talking about negative issues. For example, "I read this to say . . . could it be changed to be more meaningful?" comes across better than, "You must change this. . . ."

Jurgen Appelo's "Feedback Wrap" workout (Appelo, 2013) describes ways to give constructive feedback that helps build trust within the team. As Jurgen notes, giving feedback gets harder to do as so many teams have less face time with coworkers, due to distributed teams, remote working, and flexible hours. He points out that written feedback, done in an honest and friendly way, allows you to think more carefully and present observations and feelings in a more balanced way. It can be done fast and frequently. Experiment with different ways to provide timely feedback to team members, and keep inspecting and adapting your feedback process.

Empathy is essential for providing good feedback. Think about how you would want to receive unwelcome information. Toastmasters International, a nonprofit organization for public speaking, teaches skills for giving speech evaluations, and these are transferable to providing feedback on software teams. Giving feedback and providing information are part of testing, and there are many opportunities to practice. In

collaboration with stakeholders, identify ways to measure how successful a new feature might be. Make testing visible and transparent. If your customers always know the team's current status and trust that they'll be told of any risks or issues, they'll be more receptive to any news (good or bad) you have to deliver. Testers learn constructive ways to provide feedback, such as showing defects to programmers, without causing offense or hurt feelings. This sensitivity applies anytime you're delivering feedback.

We've talked a lot about giving feedback. What about receiving it? Strive to understand what the message being delivered is about before you jump to conclusions. Thank the people who give you feedback for their honest input. Ask questions to clarify. Too often we, as receivers, take in the emotion and the delivery rather than the message. Receivers may also have previous experiences that could cause them to hear something different from what was actually said. Listen to learn. See the bibliography for Part II for recommended readings to learn good feedback techniques.

LEARNING THE BUSINESS DOMAIN

Domain knowledge is an example of a skill that may be either part of your broad set of skills or part of your specialty—deep and thorough. Teams with both a broad and deep understanding of the business can better help stakeholders prioritize features, simplify solutions, or even offer alternatives outside of new software to solve a problem.

Lisa's Story

Learning firsthand how customers use a software product helps us do a better job of delivering the right value to them. On my current team, testers handle customer support via email, with help from the programmers. We also monitor our product's community forum. We've had to practice our patience and tact so we can ask good questions, listen to the responses, and build a rapport with users so they feel free to share their feedback. We learn firsthand how our customers use our product and get valuable feedback on what features would provide the most value for them. We use our judgment to decide when an issue or feature request needs to be escalated. We often encounter scenarios that provide useful test cases or test charters for us to use while testing new features.

I've had a lot of experience working in customer support for a software product company. Understanding the product from the users' perspective has proven valuable for helping to build in the quality they desire.

Knowing how the business operates allows you to explore the software in the same ways real end users will use it. Not only does this prevent bugs from going out to production, but testers and other team members with domain knowledge can also give business experts ideas for new features.

Learning the Domain with Help from the Call Center

> **Mike Talks**, *a software tester from New Zealand, shares an experience he had collaborating with call center staff and learning more about how they used their product.*

A few years ago, I worked for a very delivery-focused but diverse team at a bank. What was superb about this situation was that I had access to everyone who was connected to the product.

What turned out to be really useful was having access to the call center staff. They gave me a practical introduction to how they used the product day to day. Some features and behavior I found a little surprising, but it led to me getting to know how the system actually worked. I learned to demonstrate that knowledge to marketing managers, business analysts, and product owners.

It was great to be colocated so these kinds of relationships could be built up. The relationship with the call center was really something important. I'd sometimes show them behavior on upcoming builds, but likewise they'd show me any "oddness" in production. It seems call center staff have a different mentality from testers, in that if something goes wrong, they don't want to touch it again.

It was a really rewarding team because I felt we were all contributing toward a goal, with no barriers to communication.

COACHING AND LISTENING SKILLS

A team member with good coaching skills is better able to help less experienced team members. Since everyone on an agile team engages in testing activities, it's much more valuable to guide others in solving their own testing and quality issues than to just give them the answers.

Telling stories from your own experience is a powerful way to demonstrate how something can be done. It depersonalizes criticism and allows people to remember how you handled a specific problem; they can learn to adapt your solution to their context. Think about how you have handled certain situations and how you can apply that experience to your current context. Storytelling doesn't come naturally to many people; it takes practice. See the Part II bibliography for references on coaching.

Observing and listening (see Figure 4-3) are critical communication and collaboration skills. Is someone complaining? Maybe it's a legitimate complaint. Does a teammate have an idea but feels too shy to speak up? Give her a comfortable opportunity to share with you over coffee. Knowing when and how to listen helps the team grow and improve. As

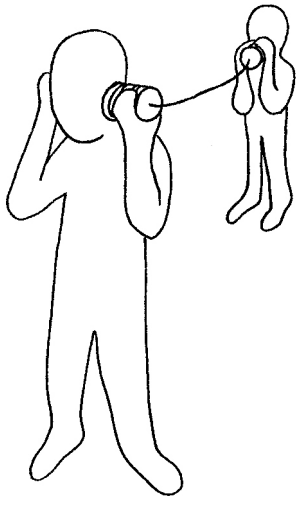

Figure 4-3 Listening—seeking to understand

Naomi Karten has pointed out (Karten, 2009), listening is a sometimes-overlooked component of being a good collaborator.

Thinking Differently

We've discussed several thinking skills that we find helpful as we engage in testing activities. Be conscious of the style of thinking you adopt for each situation. The next story explains some thinking tools and styles.

Thinking for Testing

Sharon Robson, *a software testing practice lead from Australia, shared her experiences with ways people tasked with testing can maximize their time by applying basic thinking skills.*

Thinking is a skill that can be learned and improved. As with any skill, practice and exposure to new techniques and approaches can hone an innate or organic talent. By learning about and practicing new ways of thinking, basic skills can be sharpened and new skill levels exposed. We all think, but thinking constructively, with purpose and the goal of achieving the desired outcomes, is an ever-evolving skill set.

There are many different ways of thinking, such as critical thinking (examining the accuracy of a statement), analytical thinking (decomposing the subject into component parts and considering them and their relationships), and creative thinking (synthesizing new knowledge from existing data). Testers need to recognize the skill needed in each situation and pick and choose the tools that will assist them in gaining the desired outcome.

There are many different thinking tools that are available too, such as Socratic questioning (Wikipedia, 2014k), functional decomposition, prioritization, compare and contrast, Ishikawa diagrams for the more critical or analytical approaches, as well as the creative thinking approaches such as brainstorming, mind mapping, elaboration, and relationship mapping.

In *Pragmatic Thinking and Learning* (Hunt, 2008), Andy Hunt discusses the concept of L-Mode (linear and slow) and R-Mode (nonlinear, fast, "rich" mode) and how to employ these modes when needed. Tools such as working to define SMART (specific, measurable, achievable, relevant, and time-boxed) goals help people realize what mode they need to be in and then employ other tools to move to the right mode to maximize effective thinking. One of Hunt's most powerful tools is

called the "beginner's mind"—keep asking, "What if . . . ?" a lot! Hunt also advocates taking time to see and understand clearly what is happening. He cleverly articulates that information is raw data. Knowledge imparts meaning to the information, and context allows true understanding. As testers, we should focus on gathering true understanding and not stop at information or knowledge. Much of a person's perception is based on prediction, and prediction is based on context and past experience. From the viewpoint of a tester, this is very powerful as our perceptions could be clouded and not reflect reality. Testers need to use their thinking skills to move beyond pure perception into evidence-based results.

The key is to recognize the type of thinking required and make the effort to go to the right thinking mode to meet the needs of the situation. There are a variety of works on thinking that cover these various thinking modes. Daniel Kahneman (Kahneman, 2011), in *Thinking, Fast and Slow*, discusses System 1 (intuitive, fast) and System 2 (systematic, slow) and talks about aspects that are important to testers, such as recognizing cognitive bias or the use of heuristics as a risk, and then methods that can be used to move from System 1 to System 2 to quantify and establish evidence of the information provided by System 1.

Edward de Bono (de Bono, 1998), in *Thinking for Action*, discusses the tools that can be used to enhance how testers look at and consider the information presented to them. Techniques such as OPV (other people's views) and Logic Bubbles (assuming everyone is very intelligent and that they think and act logically from their perspectives) can allow testers to identify when and where defects may occur. Professor de Bono also uses the "hypothesize and then prove it" approach—make a guess, then find the evidence to prove or disprove the guess. He focuses a lot on the mindset for testing in the types of information-gathering tools he introduces—checking (yes/no answers) versus exploring (asking open questions).

Elisabeth Hendrickson (Hendrickson, 2013) uses some fabulous tools for structured and focused thinking in *Explore It!*. She discusses using diagrams and maps to consider the solution from different aspects, and the heuristics of "always/never," "inverting the result," and "nouns and verbs" to focus the testing mind on the correct things. These tools are very useful in changing the way we think, which we often have to do.

Dan Ariely (Ariely, 2008) discusses the decisions we make and how as humans we tend toward the default position. As testers it is important

that we recognize the default position and check our tendencies and those of the people using our solutions to be swayed (fairly or unfairly) by the options they are presented with. This can manifest in many ways, even cognitive bias (we agree with findings/evidence that supports our position and can fail to see things that refute our position).

Depending on what they are doing, testers may need to recognize their mode and make an effort (using tools) to move to a new mode.

See the bibliography for Part II for references to learn more about the ideas Sharon mentions.

ORGANIZING

Time is always a constraint, so if we're going to accomplish essential testing activities, good organizational skills are vital. Knowing how to plan and manage your time well, using approaches such as risk-based testing, can help you focus on the right tasks. With so many demands on time, including meetings, email, instant messages, planning, and tracking activities, it can be hard to do actual testing. It's too easy to end up thrashing, repeating the same tests over and over. Knowing how to organize your time also helps ensure that you have time to learn any other skills your project may require.

Janet's Story

In Chapter 3, "Roles and Competencies," we talked about competencies and strengths. I'll share a personal story about one of my strengths and how we incorporated it into the process of writing the book. We have deadlines that we agreed to with the publisher. Using my organizational skills, I created the release plan and kept it visible so Lisa and I could talk about risks and plan accordingly. I created the shared spreadsheet to keep track of who gave us stories and when they were updated. These simple tools helped to keep us on track; organization is essential in almost everything we do.

Of course, I have weaknesses, too. For example, wordsmithing is not my strength, so often when I have an idea, I convey it the best way I can and then have Lisa work her magic.

COLLABORATING

Be sensitive to the downsides of task switching as you plan each day. And if you're feeling overwhelmed, don't be afraid to ask for help. Agile testing needs to be a collaborative effort.

An Effective Collaboration Process

We've discussed a wide range of thinking and interpersonal skills in this chapter. **Sharon Robson** *pulls many of these together to show how they can help teams collaborate for more effective testing.*

One of the key goals of an agile team (ideally any team) is to collaborate on the work they are doing. Collaboration adds to the quality of the outputs and the buy-in of the team working together. However, collaboration is hard! To collaborate well, team members need to understand why they are collaborating and then plan how they will collaborate effectively and efficiently.

The Collaboration Process

All these steps need to be done as a team for each collaboration session. Each session should be focused on one goal and ideally not exceed one hour. Some sessions will need more process definition than others.

1. Define the goal of the session—ensure that there is a clear and specific outcome, for example, to define the XYZ stories, to elaborate story 57, to coordinate our work for the day. Once the goal is clearly understood and agreed, document it simply and move on to the next step.
2. Define the language to be used. Each session will use specific words that have contextual meaning; within the context of the session the meaning must be clearly understood by all collaborators. Any unclear, ambiguous, or confusing words should have definitions or assumptions made about them to enable the team to discuss step 3.
3. Define the process to achieve the goal (e.g., brainstorming, chartering, discussion, diagramming) based on the goal of the session. What activities need to be completed to achieve the goal? These are usually in the form of a discovery or investigation stage (brainstorming), then an analysis stage (grouping, discussing).

These lead to an understanding or evaluation stage (diagramming or mapping), followed by a conclusion or decision stage.

4. Set the time boxes for the process stages. Once all the stages have been set out, allocate times to each of them that fit within the overall time box. *Note:* Allow time for rework!
5. Follow the process, adding to or tweaking the process, assumptions, and language as you go. Keep focusing on the goal! Keep asking yourself if the current activity is driving toward the goal.
6. Assess the goal. Has it been met? If yes, conclude the session; if no, change something.
7. Repeat steps 2 through 6 as necessary until the goal has been met.

The collaborative sessions Sharon describes require competent facilitation. Even if your team has an experienced facilitator, understanding meeting dynamics and learning facilitation skills will help everyone on the team get more value from meetings and collaboration sessions.

SUMMARY

Thinking skills play a big part in all aspects of software testing. Some of the most important ones to practice include

- Facilitating
- Problem solving
- Giving and receiving feedback
- Learning the business domain
- Coaching and listening skills
- Thinking differently, and applying different styles of thinking to different testing activities
- Organizing
- Collaborating, using a step-by-step process

This page intentionally left blank

Chapter 5

TECHNICAL AWARENESS

5. Technical Awareness
- Guiding Development with Examples
- Automation and Coding Skills
- General Technical Skills
- Development Environments
- Test Environments
- Continuous Integration and Source Code Control Systems
- Testing Quality Attributes
- Test Design Techniques

Thinking skills help the whole team work well together to ensure that all necessary testing activities are planned and executed. Technical testing skills help bridge the gap between what the business needs and how the delivery team supplies it. We've used the term *technical awareness* to cover the ideas of technical skills needed for testing and communicating with other members of the development team. The first time Janet heard the term *technical awareness* was at a local testing workshop where Lynn McKee used it during one of the discussions. It seemed to capture the intent without getting hung up on existing terms. We believe that testing is in itself a specialist technical skill, so we've decided to devote this chapter to many types of technical skills that will be useful in your agile testing toolkit.

GUIDING DEVELOPMENT WITH EXAMPLES

We have conversations with customers about examples of desired and undesired behavior for each new feature and story. These examples can be turned into automated business-facing tests that guide development. Some well-known approaches for this are specification by example (SBE), acceptance-test-driven development (ATDD), and behavior-driven development (BDD). If you're not yet familiar with

these, check the bibliography for Part II, "Learning for Better Testing," for some good places to start learning. *ATDD by Example* by Markus Gärtner (Gärtner, 2012), *Specification by Example* by Gojko Adzic (Adzic, 2011), and *The Cucumber Book* by Matt Wynne and Aslak Hellesøy (Wynne and Hellesøy, 2012) are good introductions. Capturing appropriate examples and turning them into automated tests requires a certain amount of technical awareness to be able to collaborate with your programmers. We'll talk in more detail about this practice in Part IV, "Testing Business Value."

AUTOMATION AND CODING SKILLS

When testers collaborate with programmers, system administrators, database experts, and people in other technical roles, they can help each other design effective tests at all levels and automate day-to-day tasks such as deploying code to test environments. This collaboration is easier when testers have some technical knowledge and all development team members have some testing skills.

If testers learn to use the same integrated development environment (IDE) as the coders, pairing to look at code becomes easier, and the language used to discuss the application becomes a shared language.

Tests may be specified in a natural style, using a domain-specific language (DSL), or by using keywords or data to guide the tests. However, there is underlying code that executes the tests and produces the results. Someone has to write that, and test automation code is, well, code. If you, as a tester, do not know how to code, it is important to be able to understand what the code does.

When a team guides development with business examples and practices test-driven development (TDD), the tests help create good code. They provide living documentation of how the production code behaves and ensure that it keeps behaving that way. If the code needs changing later, the tests make doing so safer and faster. Therefore, we should treat test code with the same care and respect as production code—they are equally valuable.

Applying good design practices helps keep automation cost-effective. For example, we strive to keep each automated test focused on a single

clear purpose and extract duplication into macros and modules. Our goal is to make it easy to diagnose test failures and change the tests in only one place when the code changes. You can find more detail about that in Chapter 16, "Test Automation Design Patterns and Approaches."

Learning how to write pseudo code can help you design automated tests. Gain a basic understanding of object-oriented principles like SOLID (Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion) (see Wikipedia, 2014m).

If you know how to read the code and understand your team's coding standards, you may be able to pair with programmers to write tests, review code, and perhaps help with debugging problems. At the very least, you'll be able to have meaningful conversations with the programmers on your team to learn which areas of the code are most fragile and where to focus testing efforts. This is also an opportunity to improve your coding and scripting skills. As a side benefit, you might uncover defects while you're pairing.

If you don't have programming experience, practice some basics on your own. Check the bibliography for Part II for helpful books on learning to code, such as *Everyday Scripting with Ruby* by Brian Marick (Marick, 2007). There are also online courses, tutorials, and screencasts.

Coding skills are also useful to help set up data and scenarios for exploratory testing charters. Even if you lack programming skills, knowing what could be automated will help you collaborate with programmers on your team to do time-saving automation and free testers up for activities where they contribute the most value.

Learn your software product's architecture, at least at a high level. Ask teammates to identify the risky areas. Lisa's team represents more fragile areas of the architecture with dotted lines in diagrams. This visibility helps you to focus on different types of testing effectively and consider test automation strategies together with your team. Systems design knowledge can help you test efficiently. For example, if you know that a search function used throughout the application is encapsulated in one part of the code, you may be able to test it thoroughly via an API and need only cursory checks in other parts of the application.

Understanding the system interfaces lets you recognize their possible weaknesses. There is more than the user interface (UI); watch for others such as logging and monitoring for operations, messaging, or communication protocols.

Figure 5-1 shows an example of the type of architecture diagram that provides a good starting point for understanding the components of a system and how they relate to each other. This one shows the architecture for an API that Lisa's team developed. It helped the team visualize how the software would generate user documentation and request validation data as well as execute commands to add, change, or delete data resources.

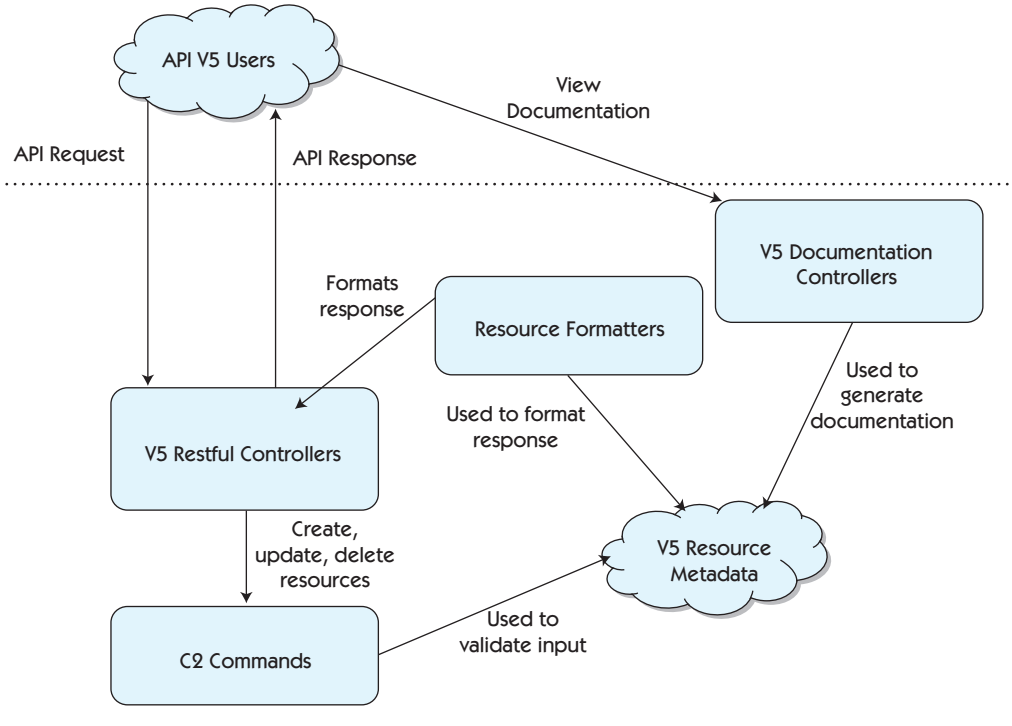

Figure 5-1 Example architecture diagram for an API

General Technical Skills

Every team and organization we work with has different needs and different contexts. The technical skills that are necessary to test effectively vary from team to team. However, there are some basic technical skills that will serve you well in any environment.

Knowing how to monitor processes, memory, and CPU, read log files, and interpret profiling statistics can help you identify potential resource misuse or leaks. Also watch for and understand important performance characteristics that might otherwise go unnoticed until your product gets into production. Using monitoring and logging is another area where a tester and programmer or DevOps practitioner can pair to really dig into elusive problems. The ability to view or tail a log file while testing is a necessity with most software products. Being comfortable on a command line with basic UNIX shell commands is useful for a wide range of activities, such as maintaining test environments, monitoring log files, and testing APIs.

Most testing requires at least some database knowledge. Checking data is only one aspect of testing. We also need to verify relational integrity and constraints. Basic SQL or other query language skills are a must for testing any application that uses a relational database for persisting data. Pairing a database expert with a tester is an excellent way to verify quality attributes of the database, while transferring skills. We will address testing data and databases more in Chapter 22, "Agile Testing for Data Warehouses and Business Intelligence Systems."

Development Environments

The ability to update, build, and deploy your team's latest code to your local machine provides more flexibility in testing and debugging issues. If you're helping to automate tests, you'll probably want to check out the latest code, write the tests, run them locally, and check in the new tests. Of course, this depends on your context. For example, Adam Knight told us that his team pulls the build from the continuous integration (CI) system artifacts and executes multiple test runs in parallel so the testers have a massive automation effort with no involvement in building the code.

Another benefit of being able to see the latest code is that we can often save time by doing some of the "fixes" ourselves. When Lisa sees a spelling mistake in a help page, she can fix it herself, run the automated tests, and check in the fix. Whatever checks and balances are in place for your programmers apply to testers as well. No matter what your role, if you are touching code, you need to follow coding standards and practices and ensure that appropriate testing is done.

In order to do this type of work, testers need to get up to speed on the source code control tool that their team uses and, ideally, the IDE. This is something that is easily done with tester/programmer pairing. When Lisa pairs with a teammate to work on code, she makes notes of techniques she might need later and keeps them on the team wiki for future reference and to share with other testers.

TEST ENVIRONMENTS

Many agile teams struggle with creating and maintaining useful test environments. Some teams configure their CI builds to automatically deploy build artifacts to test environments when test suites pass. In other teams, testers deploy the artifacts they want to test when they are ready. Sadly, some teams still don't have a CI process, and some don't even have adequate test environments. Without a dependable build process, it is hard to give fast feedback to the team on stories delivered. Without adequate test environments, it is hard to supply correct information on the state of the code.

A standard practice we've seen work in many teams is to have several test environments for different purposes. Programmers generally use a local sandbox that they maintain themselves for testing purposes. Each tester may also have her own local test environment. Ideally, a team has at least one test environment that mimics the production system for realistic exploratory testing. Teams also need a staging environment that's a copy or at least a good representation of production where they can test each release, including any data migrations. A common practice is to copy a snapshot of production data, with any sensitive data scrubbed, into the staging environment, so you're testing with realistic data.

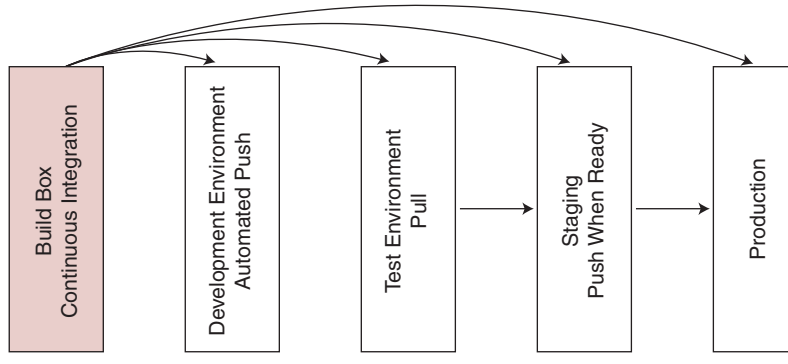

Figure 5-2 Simple build pipeline

There can be many more test environments, such as one specialized to measure load and performance, or one to run the automated test suites. Figure 5-2 represents the simplest form of a build pipeline, but these can be complex. In Chapter 23, "Testing and DevOps," a contributor shares how he managed his test environments.

Teams that have multiple code bases or branch their code from the master (main development trunk) source code control version generally need additional test environments. Lisa's team has test servers that are used to test architectural or code design spikes, or special projects that aren't yet part of the production code. These are also useful when upgrading to a new version of the programming language or development framework.

Testers should learn about all the different test environments and know when to ask for different configurations or additional servers. In our experience, testing is most effective when testers know what code and data should be on a given test environment and have the skills to ensure that it is there. Janet worked with a team that listed all the test environments on a large whiteboard. It showed exactly what version of the build was on which environment and when it had been deployed. A quick visual check ensured that the team knew what they were testing. Lisa's

team documents the various test and staging environments, including the purpose of each one, in a spreadsheet on the team wiki.

CONTINUOUS INTEGRATION AND SOURCE CODE CONTROL SYSTEMS

As teams grow, so do the complexity and the number of CI builds and test suites. Teams start encountering the dilemma where the unit tests pass, but the tests for a particular browser, or an integration test suite, fail. We need to question, "Is this code worthy of testing in any environment?" It is important to have a common understanding of what it means to the team to get all the tests passing—getting to green—again.

Take time to understand your team's CI configurations. For example, if your CI process deploys automatically to a test environment, make sure you know which test suites must pass before the build can deploy. Testers need to learn what the different CI jobs are and how the success or failure of each relates to testing. If you're testing the business logic on the server side, it may not matter that a particular browser test suite failed. But if you're testing client-side logic, you may prefer to wait for all regression tests on all browsers to pass.

Understand what the risk is for your system. Know which programmers to talk to if a particular test suite is failing. Sometimes the tests that are failing don't affect the particular feature you want to test, so it may be possible to manually deploy the code and continue testing.

Having multiple development teams complicates the CI process. What we've seen most often is that individual teams have their own CI process for their code, and most testing can be done in the team's own test environment. Their code is also tested as part of a company-wide CI process. Regression failures in other teams' code could affect your ability to test the larger system. If there isn't already a process to identify the source of failures in a company-wide CI and some means to get the responsible team to fix the problem, try to get the right people together to put one in place to determine how to solve the issue.

Branching complicates CI issues. If your team always stays on the master (sometimes called main or trunk), life is simple. But let's say your

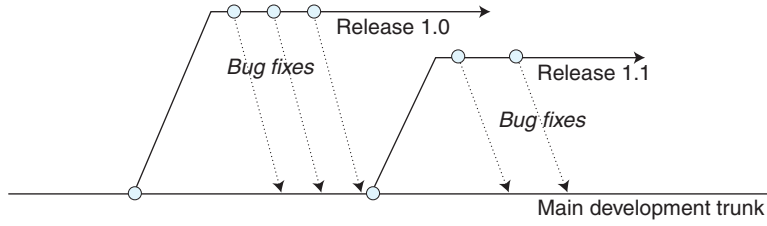

Figure 5-3 A possible branching strategy

team is doing a big refactor on master, while a bug fix needs to go out on the production branch. You need to be sure to check out the right thing and/or deploy the correct build to the appropriate test environment for verification. This is another area where collaborating with programmers and possibly DevOps is essential. In the diagram shown in Figure 5-3, the bug fixes are either merged into the main development trunk or double-punched (both updated with the same fix).

A Simple Branching Strategy

Augusto Evangelisti, *an agile testing enthusiast from Ireland, shares his story about how his organization handles branching in the simplest way possible.*

We have resolved the issue around branching and merging quite drastically by removing branching altogether and doing pure continuous integration, with everyone coding directly on trunk. We are lucky that we have only two or three teams pushing at any one time, as well as a very advanced and fast CI system that makes it very smooth. We have recently introduced feature toggling because we are doing continuous delivery and sometimes our customers are not ready to use a new user story. We test it in our internal environments, but we toggle it in production through Spring configuration.

We use Git, a distributed source code management tool, and each developer's box is a "branch" as such until he or she checks in the code.

Ideally, we could all work on the master of our source code, and many organizations are able to succeed with that strategy. However, as organizations grow or perhaps have more complicated systems, other strategies evolve.

A More Complex Branching Strategy

> **Adam Knight**, *a director of QA and support from the UK, shares his team's branching experiences, which are completely different from Augusto's and are much more complex.*

Because we are a database product company with a traditional delivery model of supported software releases, branching and merging are unfortunate facts of life. As our customers implement our product on-site as part of major installations, we need to maintain branches of previous releases that are still in production on customer sites around the world.

We use Subversion and keep the tests along with the code. At the point of each release, we branch the tests along with the code onto a release branch for that version. Should any major fixes be required, we apply these to the trunk code and test. We then push both the fix and the new tests back to any relevant release branches and issue update releases as required.

One of the main challenges that we face is trying to keep these release branches "linear," as often different customers require different fixes and don't necessarily want to risk having other fixes in their update releases. So far I've managed to win this game, as I believe that having separate individual patches exposes a huge amount of risk given the exponential number of untested combinations in which those patches could be applied. I have had to explain to some customers who have questioned this approach the vast difference in risk between a patch and a full version of the software that has been through all of our automated testing prior to being released.

Another situation where branches may be created is if we are involved in a proof of concept (POC) where the requirements of the POC demand additional work to be done on the code base to achieve them. In this situation we branch the code and add a high-priority prototyping story to the iteration to try to achieve the specified targets (usually around query performance or concurrency for a specific data set). Once these items are complete, we create stories for subsequent iterations for each feature that we want to bring into the main trunk.

> Those stories involve reworking the features to make them release quality and to fully integrate them with the rest of the product with appropriate testing. The major risk with this approach is that there is a perception of a higher level of completion of those prototype items than is actually present and consequently an unrealistic expectation around how quickly they can be converted into releasable code.
>
> As I said, the need to maintain multiple branches is not a desirable situation but a fact of life given our context and delivery model.

As Adam notes, testers and their development teams also must manage tests and test code within the context of the team's source code control system. Automated tests should be versioned with the code that they test. If you aren't already familiar with your team's source code control system, pair with a programmer, system administrator, or DevOps person to learn how to use it and how your team's code is organized.

Testing Quality Attributes

Agile Testing explained how to use the agile testing quadrants (Figure 5-4) to help ensure that the team plans and executes the types of

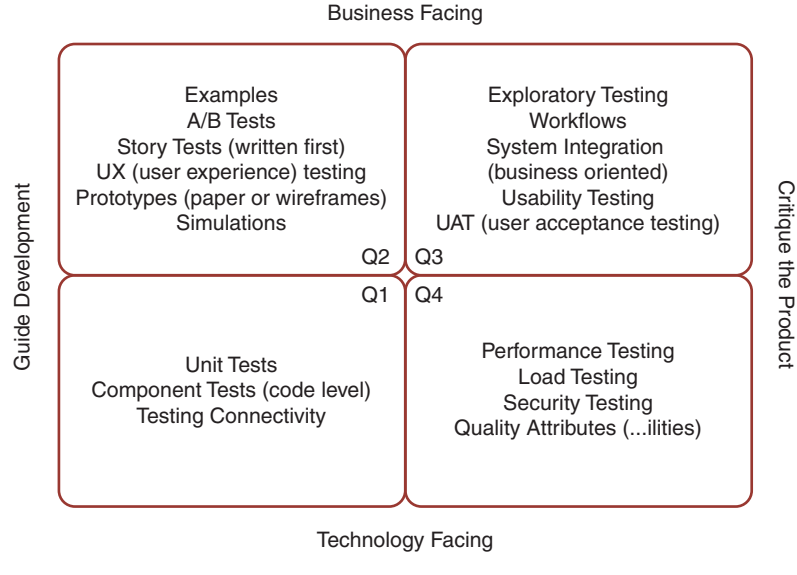

Figure 5-4 Agile testing quadrants

testing that they need. Chapter 8, "Using Models to Help Plan," goes into detail on planning for all four quadrants, but in this chapter we'll cover some of the technical abilities you may want to enhance.

The tests in Quadrant 4 (technology-facing tests that critique the product) are sometimes ignored or underestimated since they are often where testing skills are the weakest. This quadrant contains important quality attributes or constraints that are frequently not specified in the stories. An example of an unstated constraint is that "performance on all web pages must respond in less than three seconds." If your team uses analytical tools to measure performance, you will want to learn how to run them and interpret the results.

Organizations may lack people with the skills for some specialty areas. For example, maybe your team has no security testing specialists. Your team can learn some basic security testing techniques to help give your company some degree of safety. Learning about SQL injection and cross-site scripting will cover some of the basic areas of vulnerability. Janet had a recent experience with a website that wanted payment and asked for full credit card details. She noticed it was not an HTTPS site so did a bit of testing. The site didn't even hide the details in the page, and a simple "view source" after submitting showed all the details she had entered in plain text. However, there's a lot more to ensuring the security of a software product. Educating your company about the value of a security audit may be the most valuable contribution you can make.

Find out what is important to your organization and become technically aware in that area. Perhaps it is enough to know that you can recommend they hire an expert.

We place exploratory testing in Quadrant 3 (business-facing tests that critique the product), and we have devoted Chapter 12, "Exploratory Testing," to the subject. However, we wanted to call it out here because it is a skill set that you must continue to practice and grow. Teaching exploratory testing skills to programmers on your team can help them be better coders. In her book *Explore It!* (Hendrickson, 2013), Elisabeth Hendrickson gives examples of how programmers can use exploratory testing at a code level as well. Sometimes, thinking about the ripple

effects of small pieces of code will help them avoid breaking other parts of the application with new code or changes to existing code. Pairing with testers helps coders gain testing awareness.

Test Design Techniques

When you are deciding what and how to test, it is important to have a toolbox full of techniques, such as the use of state transition diagrams or decision tables. We've included recommended books and courses on test design and domain testing in the bibliography for Part II.

Summary

"T-shaped" team members need technical awareness in a variety of areas to effectively plan and execute testing activities. There's a wide variety of basic knowledge that will help team members in all roles collaborate more effectively for testing. Testers bring in-depth testing skills to the party but need to communicate effectively with programmers, system administrators, and other technical roles. At the same time, team members in other roles who engage in testing activities should also practice these skills. These include

- Guiding development with examples
- Technical awareness of architecture and code design
- Automation and coding skills
- Maintaining test environments
- Understanding source code control and continuous integration
- Testing quality attributes, guided by the agile testing quadrants
- Test design techniques

This page intentionally left blank

Chapter 6

How to Learn

In Chapter 3, "Roles and Competencies," we explained the importance of becoming a generalizing specialist who is able to collaborate with team members who have other specialties while contributing one's own deep expertise. In Chapter 4, "Thinking Skills for Testing," and Chapter 5, "Technical Awareness," we explored a variety of skills that help software teams do a better job of testing and delivering a high-quality, high-value product. Growing all these different skills broadens our perspective and lets us approach our work more creatively. In this chapter, we'll look at ways to learn these diverse skills. There are plenty of educational resources around, but first, think about your learning style and your team culture, both of which affect your success at acquiring new abilities.

Learning Styles

Each of us has preferred ways of learning. Some of us learn best by listening—we're auditory learners. Some of us like to see pictures—we have a visual preference. Many of us learn by doing—some people call that kinesthetic learning. Many times we need to absorb information in more than one way. For example, Janet is an auditory learner, but she needs to practice skills to internalize them. However, when she tries to get a concept across to others, she needs to draw it to help explain it. Different avenues of learning apply to everyone but to different degrees.

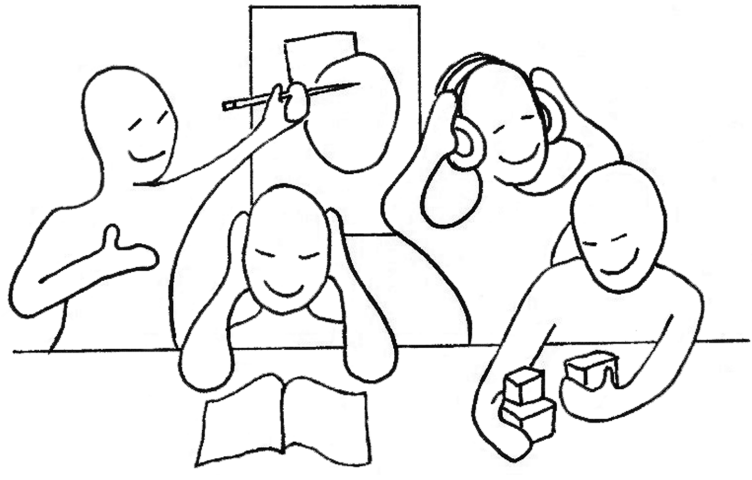

Figure 6-1 How do you learn?

What is your preferred style? Understand yours so you can get the best out of each learning experience.

Emotions affect the way we take in information. All of us have blind spots that may prevent us from learning, triggers that cause us to shut down so that we don't hear the message anymore. To learn and question, we need a safe environment. If you're helping other team members learn a new skill, keep in mind that people may have emotional reactions to what you are saying or how you are saying it. They may not be taking in the message if they don't like the delivery format. Figure 6-1 shows the many styles people have—what works for one person may not work for another.

As you're learning, whether it's a training class, online tutorial, or one-on-one session, reflect on what emotional "hot buttons" you have. When you feel yourself blocking out information because of the way it's presented, try to focus on the value you can get from the instructor or material. Keep this in mind when collaborating with developer teammates and business experts, and take in all the information that will enhance testing. Look for your own blind spots that may prevent you from learning, and observe to recognize them in others (Gregory, 2010).

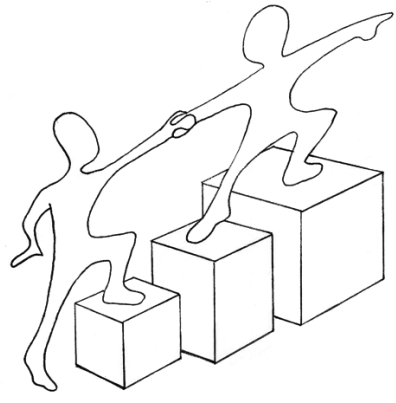

Figure 6-2 Helpers

David Hussman and Jean Tabaka facilitated a workshop called "Zen, the Beginner's Mind," designed to open minds to new ideas (Levison, 2008), at Agile 2008. Janet participated and learned that when we approach issues with closed minds and start every sentence with, "But that won't work because . . ." or, "That didn't work when we tried it in . . ." we close our minds to opportunities. Maybe it was just a small, solvable problem that made it fail before. That doesn't mean the fundamental idea was flawed. Or perhaps you weren't ready to hear it. Sometimes we don't have the basic information that will enable us to grasp a concept. Open your mind to the wonder of learning; reading books in philosophy or other studies can show you different ways to look at a problem.

Look beyond the software testing and development profession for learning opportunities. Instructors, coaches, or mentors in other fields may fit your learning style and bring a different perspective. The right helper can show us ways to reach for new ideas (see Figure 6-2).

Janet's Story

When Lisa and I started developing and teaching our agile testing course, I started to read about teaching strategies. I picked up a few tips from the books and applied them. I also watched other instructors and tried to find the style that suited me. However, one of the people I turned to the most for mentoring was my sister, Carol Vaage. She was a grade-one teacher with

her master's degree in early childhood learning, and she opened my eyes to new ways to approach teaching and working with my classes.

For example, she introduced ways of using open-ended questions to get her classes to start thinking about a topic. I've included a few here, but the list is much longer and is included in Appendix B, "Provocation Starters."

- Let's figure out how that could be . . .
- What would happen next?
- Why would that be?
- Why do you think that happened?
- Did you notice?

If we think about how children learn and give ourselves permission to explore our natural curiosity about the world, think how far we could go.

Carol also has some amazing stories and pictures of how far children can go to learn, which we've included in the bibliography for Part II, "Learning for Better Testing."

You can learn from people you admire even without a formal mentoring relationship. Most people are happy to help, so have the courage to ask questions.

LEARNING RESOURCES

Seek out places to sharpen your skills. You may find good resources online, in your local community, or farther afield. Let's look at some examples of good learning opportunities. See the Part II bibliography section "Courses, Conferences, Online Communities, Podcasts" for links to the activities mentioned in this section.

Conferences, Courses, Meet-ups, and Collaborating

A good conference can provide a variety of takeaways, ranging from specific techniques you can try right away, to groundbreaking new ideas or technologies you can continue to research. Most importantly, you'll meet practitioners and thought leaders with whom you can form a lasting network, a constant source of inspiration and ideas.

Consider different types of conferences. Testing conferences are an obvious choice for testers, but consider others, such as those that help

you work on specific skills such as scripting languages. If your employer can't afford to send you to a conference and you can't afford to pay your own way, consider proposing a paper or session to a testing or software development conference. This may earn you a free or discounted conference registration. And remember, teaching a skill to someone else is the best way to learn it yourself. Find out if the conferences need volunteers, or if you qualify for grants or discounts. If you can't travel to attend a conference, consider virtual conferences, which are becoming more popular, or sign up for webinars.

Conferences aren't only for learning specific testing and technical skills. Many software conferences have sessions and tracks on collaboration, organizational culture, learning, coaching, working with customers, and mentoring. Janet has even attended sessions by Portia Tung on the "Power of Play." Figure 6-3 shows people learning and playing together.

The most important learning at conferences often takes place at break times and in the hallways and dining areas, as you meet new people who become part of your network.

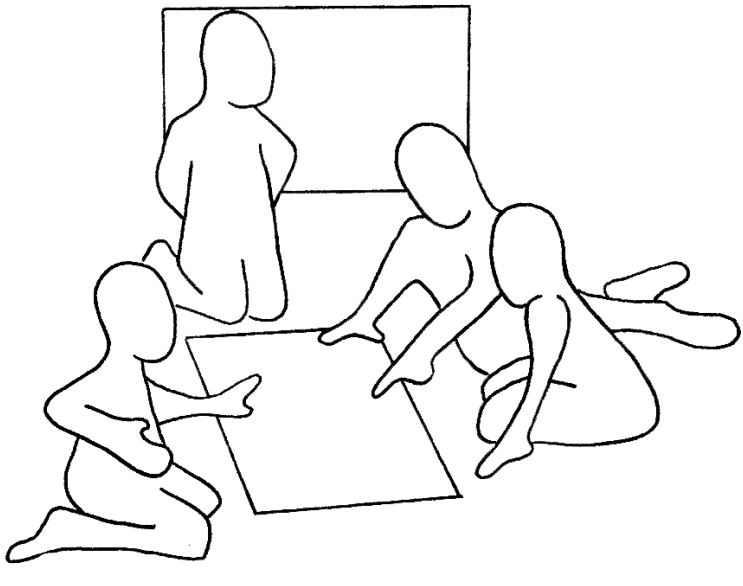

Figure 6-3 Collaborative learning and play